Iterator-Based Inventory System.

Develop an Inventory System where products can be safely removed during traversal usingan Iterator ●

Code

```
class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def __iter__(self):
        return InventoryIterator(self)


class InventoryIterator:
    def __init__(self, inventory):
        self.inventory = inventory
        self.index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.index >= len(self.inventory.products):
            raise StopIteration
```

```
        product = self.inventory.products[self.index]
        self.index += 1
        return product

    def remove(self):
        if self.index > 0:
            self.inventory.products.pop(self.index - 1)
            self.index -= 1


# Main Program
inventory = Inventory()

n = int(input("Enter number of products: "))

for _ in range(n):
    product = input("Enter product name: ")
    inventory.add_product(product)

iterator = iter(inventory)

print("\nRemoving products starting with 'A':")
try:
    while True:
        product = next(iterator)
        if product.startswith("A"):
```

```
            iterator.remove()
except StopIteration:
    pass


print("\nRemaining Products:")
for product in inventory:
    print(product)
```

Sample Input

```
5
Apple
Banana
Avocado
Orange
Mango
```

Sample Output

```
Removing products starting with 'A':


Remaining Products:
Banana
Orange
Mango
```